cnn_etape2

May 25, 2026

1 Étape 2 : Premier modèle CNN - Baseline et variations

1.1 Contexte et objectifs

Ce notebook constitue notre première expérimentation avec les CNN (Convolutional Neural Networks) pour la classification d'images. Nous travaillons sur un problème de **classification binaire** : distinguer les éléphants des autres animaux.

Dataset : 200 images (100 éléphants, 100 non-éléphants) redimensionnées en 128×128 pixels

1.2 Plan du notebook

1.2.1 Partie 1 : Baseline simple

- Créer un CNN minimal (1 Conv + 1 Pool + Dense)
- Entraîner et observer les courbes de loss/accuracy
- Évaluer les performances sur le test set

1.2.2 Partie 2 : Variations d'hyperparamètres

Tester l'impact de : - Plus d'epochs (60 au lieu de 30) - Learning rate différent (1e-4 au lieu de 1e-3) - Batch size différent (16 au lieu de 32)

1.2.3 Partie 3 : Modèle complexe (overfitting)

- Créer un CNN avec 3 blocs Conv + régularisation forte
- Observer l'overfitting sévère (pour comprendre le phénomène)

1.2.4 Partie 4 : Modèle amélioré optimisé

- Simplifier l'architecture pour notre petit dataset
- Trouver le bon équilibre entre performance et généralisation

1.2.5 Partie 5 : Analyse comparative

- Comparer tous les modèles
- Tirer des conclusions sur les meilleurs choix

1.3 1. Imports et configuration

[5]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import os
from os import listdir
from os.path import isfile, join
import random

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪accuracy_score

import tensorflow as tf
from tensorflow.keras import Input, Model
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Flatten,␣
 ↪Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

import zipfile
import os

# Fixer les graines pour reproductibilité
np.random.seed(42)
tf.random.set_seed(42)
random.seed(42)

print("TensorFlow version:", tf.__version__)
print("GPU disponible:", "Oui" if tf.config.list_physical_devices('GPU') else␣
 ↪"Non")
```

```
TensorFlow version: 2.19.0
GPU disponible: Non
```

1.4 2. Chargement et préparation des données

Important : Remplace les chemins par tes propres données

[6]:
```
# Télécharger le dataset
print("Téléchargement du dataset...")
zip_file = "Tiger-Fox-Elephant.zip"
!wget https://www.lirmm.fr/~poncelet/Ressources/Tiger-Fox-Elephant.zip

# Extraire
```

```
print("Extraction des données...")
with zipfile.ZipFile(zip_file, "r") as zip_ref:
    zip_ref.extractall("Data_Project")

# Supprimer le zip pour économiser l'espace
os.remove(zip_file)
print("Dataset prêt !")

# Configuration
IMG_SIZE = 128  # Taille des images

# Chemin vers les données
DATA_PATH = "Data_Project/Tiger-Fox-Elephant/"
CLASSES = ['elephant', 'Elephant_negative_class']

print(f"\nChemin des données : {DATA_PATH}")
print(f"Classes à classifier : {CLASSES}")
```

```
Téléchargement du dataset…
--2026-03-22 12:58:12--  https://www.lirmm.fr/~poncelet/Ressources/Tiger-Fox-
Elephant.zip
Resolving www.lirmm.fr (www.lirmm.fr)… 193.49.104.251
Connecting to www.lirmm.fr (www.lirmm.fr)|193.49.104.251|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 7605545 (7.3M) [application/zip]
Saving to: ‘Tiger-Fox-Elephant.zip’

Tiger-Fox-Elephant. 100%[===================>]   7.25M  6.97MB/s    in 1.0s

2026-03-22 12:58:13 (6.97 MB/s) - ‘Tiger-Fox-Elephant.zip’ saved
[7605545/7605545]

Extraction des données…
Dataset prêt !

Chemin des données : Data_Project/Tiger-Fox-Elephant/
Classes à classifier : ['elephant', 'Elephant_negative_class']
```

[8]:
```
def create_training_data(path_data, list_classes, img_size=128):
    """
    Charge et redimensionne les images depuis les dossiers.
    """
    training_data = []

    for classes in list_classes:
        path = os.path.join(path_data, classes)
        class_num = list_classes.index(classes)
```

```
        print(f"Chargement de la classe {classes} (label {class_num})...")

        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path, img), cv2.
 ↪IMREAD_UNCHANGED)
                new_array = cv2.resize(img_array, (img_size, img_size))
                training_data.append([new_array, class_num])
            except Exception as e:
                pass

    print(f"Total d'images chargées : {len(training_data)}")
    return training_data

def create_X_y(path_data, list_classes, img_size=128):
    """
    Crée X (images) et y (labels) à partir des données.
    """
    # Récupération des données
    training_data = create_training_data(path_data, list_classes, img_size)

    # Mélange aléatoire
    random.shuffle(training_data)

    # Séparation X et y
    X = []
    y = []
    for features, label in training_data:
        X.append(features)
        y.append(label)

    X = np.array(X).reshape(-1, img_size, img_size, 3)
    y = np.array(y)

    return X, y
```

[9]:
```
# Chargement des données
X, y = create_X_y(DATA_PATH, CLASSES, IMG_SIZE)

print("\n=== Informations sur les données ===")
print(f"X.shape: {X.shape}")
print(f"y.shape: {y.shape}")
print(f"Distribution des classes:")
unique, counts = np.unique(y, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  Classe {u}: {c} images ({c/len(y)*100:.1f}%)")
```

```
Chargement de la classe elephant (label 0)…
Chargement de la classe Elephant_negative_class (label 1)…
Total d'images chargées : 200

=== Informations sur les données ===
X.shape: (200, 128, 128, 3)
y.shape: (200,)
Distribution des classes:
  Classe 0: 100 images (50.0%)
  Classe 1: 100 images (50.0%)
```

[10]:
```python
# Visualisation de quelques exemples
def plot_examples(X, y, n_examples=16):
    plt.figure(figsize=(12, 12))
    for i in range(min(n_examples, len(X))):
        plt.subplot(4, 4, i+1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
        # Convertir BGR en RGB pour affichage
        img_rgb = cv2.cvtColor(X[i], cv2.COLOR_BGR2RGB)
        plt.imshow(img_rgb)
        plt.xlabel(f'Classe {y[i]}')
    plt.tight_layout()
    plt.show()

plot_examples(X, y, 16)
```

[11]:
```
# Normalisation des images
X = X.astype('float32')
X = X / 255.0

print("Images normalisées entre 0 et 1")
print(f"Min: {X.min()}, Max: {X.max()}")
```

```
Images normalisées entre 0 et 1
Min: 0.0, Max: 1.0
```

[12]:
```
# Division train/test
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

print(f"X_train: {X_train.shape}")
print(f"X_test: {X_test.shape}")
print(f"y_train: {y_train.shape}")
print(f"y_test: {y_test.shape}")
```

```
X_train: (160, 128, 128, 3)
X_test: (40, 128, 128, 3)
y_train: (160,)
y_test: (40,)
```

1.5 3. Fonctions utilitaires pour visualisation

[14]:
```
def plot_history(history):
    """
    Affiche les courbes de loss et accuracy.
    """
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Loss
    axes[0].plot(history.history['loss'], label='Train Loss', linewidth=2)
    if 'val_loss' in history.history:
        axes[0].plot(history.history['val_loss'], label='Val Loss', linewidth=2)
    axes[0].set_title('Loss pendant l\'entraînement', fontsize=14)
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Accuracy
    axes[1].plot(history.history['accuracy'], label='Train Accuracy',␣
 ↪linewidth=2)
    if 'val_accuracy' in history.history:
        axes[1].plot(history.history['val_accuracy'], label='Val Accuracy',␣
 ↪linewidth=2)
    axes[1].set_title('Accuracy pendant l\'entraînement', fontsize=14)
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('Accuracy')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
```

```
    plt.show()

def plot_predictions(model, X, y, n_examples=16):
    """
    Affiche des prédictions avec images réelles et prédites.
    """
    # Prédictions
    y_pred_proba = model.predict(X[:n_examples], verbose=0)
    y_pred = (y_pred_proba >= 0.5).astype(int).ravel()

    plt.figure(figsize=(15, 15))
    for i in range(n_examples):
        plt.subplot(4, 4, i+1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)

        # Afficher l'image (dénormaliser pour visualisation)
        img = (X[i] * 255).astype(np.uint8)
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.imshow(img_rgb)

        # Couleur selon si correct ou pas
        color = 'green' if y_pred[i] == y[i] else 'red'
        plt.xlabel(f'Réel: {y[i]} | Prédit: {y_pred[i]} ({y_pred_proba[i][0]:.
↪2f})',
                   color=color, fontsize=10)

    plt.tight_layout()
    plt.show()

def plot_confusion_matrix(y_true, y_pred, labels=[0, 1]):
    """
    Affiche la matrice de confusion.
    """
    cm = confusion_matrix(y_true, y_pred, labels=labels)

    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=labels, yticklabels=labels,
                cbar_kws={'label': 'Count'})
    plt.title('Matrice de Confusion', fontsize=14)
    plt.ylabel('Vraie classe')
    plt.xlabel('Classe prédite')
    plt.tight_layout()
    plt.show()
```

2 BASELINE : CNN Minimal

Architecture simple : - 1 Conv2D - 1 MaxPooling - Flatten - 1 Dense (sortie)

[15]:
```
def create_baseline_model(input_shape=(128, 128, 3)):
    """
    CNN Baseline minimal.
    """
    inputs = Input(shape=input_shape, name='input')

    # Convolution
    x = Conv2D(32, kernel_size=(3, 3), activation='relu', name='conv1')(inputs)

    # MaxPooling
    x = MaxPooling2D(pool_size=(2, 2), name='pool1')(x)

    # Flatten
    x = Flatten(name='flatten')(x)

    # Sortie
    outputs = Dense(1, activation='sigmoid', name='output')(x)

    model = Model(inputs=inputs, outputs=outputs, name='CNN_Baseline')
    return model
```

[16]:
```
# Créer le modèle
model_baseline = create_baseline_model(input_shape=(IMG_SIZE, IMG_SIZE, 3))

# Compiler
model_baseline.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Afficher la structure
model_baseline.summary()
```

```
Model: "CNN_Baseline"
```

Layer (type)	Output Shape	Param #
input (InputLayer)	(None, 128, 128, 3)	0

```
 conv1 (Conv2D)                  (None, 126, 126, 32)              896

 pool1 (MaxPooling2D)            (None, 63, 63, 32)                  0

 flatten (Flatten)               (None, 127008)                      0

 output (Dense)                  (None, 1)                     127,009


Total params: 127,905 (499.63 KB)

Trainable params: 127,905 (499.63 KB)

Non-trainable params: 0 (0.00 B)
```

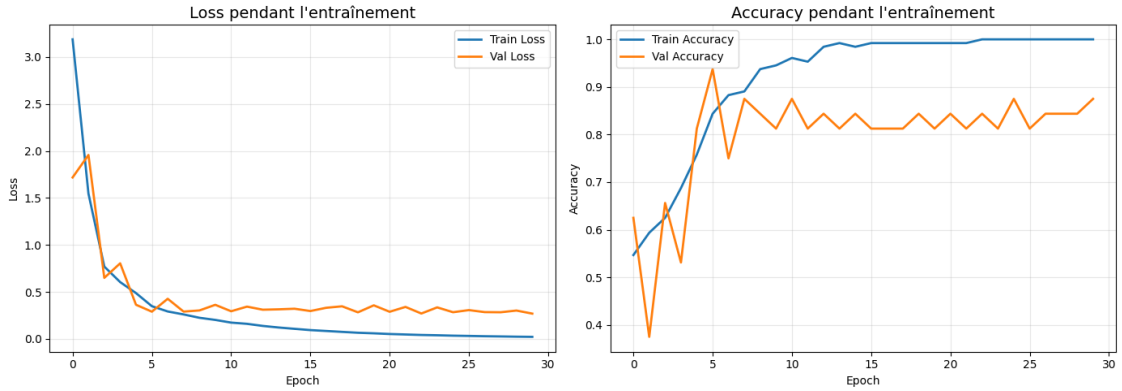
[17]:
```
# Entraîner
history_baseline = model_baseline.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/30
4/4             2s 381ms/step -
accuracy: 0.5469 - loss: 3.1882 - val_accuracy: 0.6250 - val_loss: 1.7180
Epoch 2/30
4/4             1s 364ms/step -
accuracy: 0.5938 - loss: 1.5458 - val_accuracy: 0.3750 - val_loss: 1.9574
Epoch 3/30
4/4             2s 551ms/step -
accuracy: 0.6250 - loss: 0.7690 - val_accuracy: 0.6562 - val_loss: 0.6490
Epoch 4/30
4/4             1s 326ms/step -
accuracy: 0.6875 - loss: 0.6039 - val_accuracy: 0.5312 - val_loss: 0.8038
Epoch 5/30
4/4             1s 319ms/step -
accuracy: 0.7578 - loss: 0.4866 - val_accuracy: 0.8125 - val_loss: 0.3625
Epoch 6/30
4/4             1s 314ms/step -
accuracy: 0.8438 - loss: 0.3475 - val_accuracy: 0.9375 - val_loss: 0.2891
Epoch 7/30
4/4             1s 332ms/step -
accuracy: 0.8828 - loss: 0.2912 - val_accuracy: 0.7500 - val_loss: 0.4262
```

```
Epoch 8/30
4/4            1s 318ms/step -
accuracy: 0.8906 - loss: 0.2597 - val_accuracy: 0.8750 - val_loss: 0.2900
Epoch 9/30
4/4            1s 330ms/step -
accuracy: 0.9375 - loss: 0.2240 - val_accuracy: 0.8438 - val_loss: 0.3020
Epoch 10/30
4/4            1s 321ms/step -
accuracy: 0.9453 - loss: 0.2013 - val_accuracy: 0.8125 - val_loss: 0.3619
Epoch 11/30
4/4            1s 336ms/step -
accuracy: 0.9609 - loss: 0.1726 - val_accuracy: 0.8750 - val_loss: 0.2935
Epoch 12/30
4/4            2s 542ms/step -
accuracy: 0.9531 - loss: 0.1597 - val_accuracy: 0.8125 - val_loss: 0.3427
Epoch 13/30
4/4            2s 431ms/step -
accuracy: 0.9844 - loss: 0.1373 - val_accuracy: 0.8438 - val_loss: 0.3099
Epoch 14/30
4/4            2s 323ms/step -
accuracy: 0.9922 - loss: 0.1204 - val_accuracy: 0.8125 - val_loss: 0.3140
Epoch 15/30
4/4            1s 331ms/step -
accuracy: 0.9844 - loss: 0.1065 - val_accuracy: 0.8438 - val_loss: 0.3203
Epoch 16/30
4/4            1s 333ms/step -
accuracy: 0.9922 - loss: 0.0928 - val_accuracy: 0.8125 - val_loss: 0.2958
Epoch 17/30
4/4            1s 312ms/step -
accuracy: 0.9922 - loss: 0.0835 - val_accuracy: 0.8125 - val_loss: 0.3304
Epoch 18/30
4/4            1s 338ms/step -
accuracy: 0.9922 - loss: 0.0739 - val_accuracy: 0.8125 - val_loss: 0.3461
Epoch 19/30
4/4            1s 325ms/step -
accuracy: 0.9922 - loss: 0.0646 - val_accuracy: 0.8438 - val_loss: 0.2822
Epoch 20/30
4/4            1s 370ms/step -
accuracy: 0.9922 - loss: 0.0586 - val_accuracy: 0.8125 - val_loss: 0.3565
Epoch 21/30
4/4            2s 586ms/step -
accuracy: 0.9922 - loss: 0.0512 - val_accuracy: 0.8438 - val_loss: 0.2881
Epoch 22/30
4/4            1s 337ms/step -
accuracy: 0.9922 - loss: 0.0461 - val_accuracy: 0.8125 - val_loss: 0.3396
Epoch 23/30
4/4            1s 324ms/step -
accuracy: 1.0000 - loss: 0.0410 - val_accuracy: 0.8438 - val_loss: 0.2694
```

```
Epoch 24/30
4/4             1s 342ms/step -
accuracy: 1.0000 - loss: 0.0378 - val_accuracy: 0.8125 - val_loss: 0.3347
Epoch 25/30
4/4             1s 331ms/step -
accuracy: 1.0000 - loss: 0.0335 - val_accuracy: 0.8750 - val_loss: 0.2832
Epoch 26/30
4/4             1s 322ms/step -
accuracy: 1.0000 - loss: 0.0307 - val_accuracy: 0.8125 - val_loss: 0.3058
Epoch 27/30
4/4             1s 323ms/step -
accuracy: 1.0000 - loss: 0.0275 - val_accuracy: 0.8438 - val_loss: 0.2843
Epoch 28/30
4/4             1s 320ms/step -
accuracy: 1.0000 - loss: 0.0253 - val_accuracy: 0.8438 - val_loss: 0.2827
Epoch 29/30
4/4             1s 335ms/step -
accuracy: 1.0000 - loss: 0.0227 - val_accuracy: 0.8438 - val_loss: 0.3010
Epoch 30/30
4/4             2s 617ms/step -
accuracy: 1.0000 - loss: 0.0209 - val_accuracy: 0.8750 - val_loss: 0.2683
```

[18]:
```python
# Visualiser les courbes
plot_history(history_baseline)
```

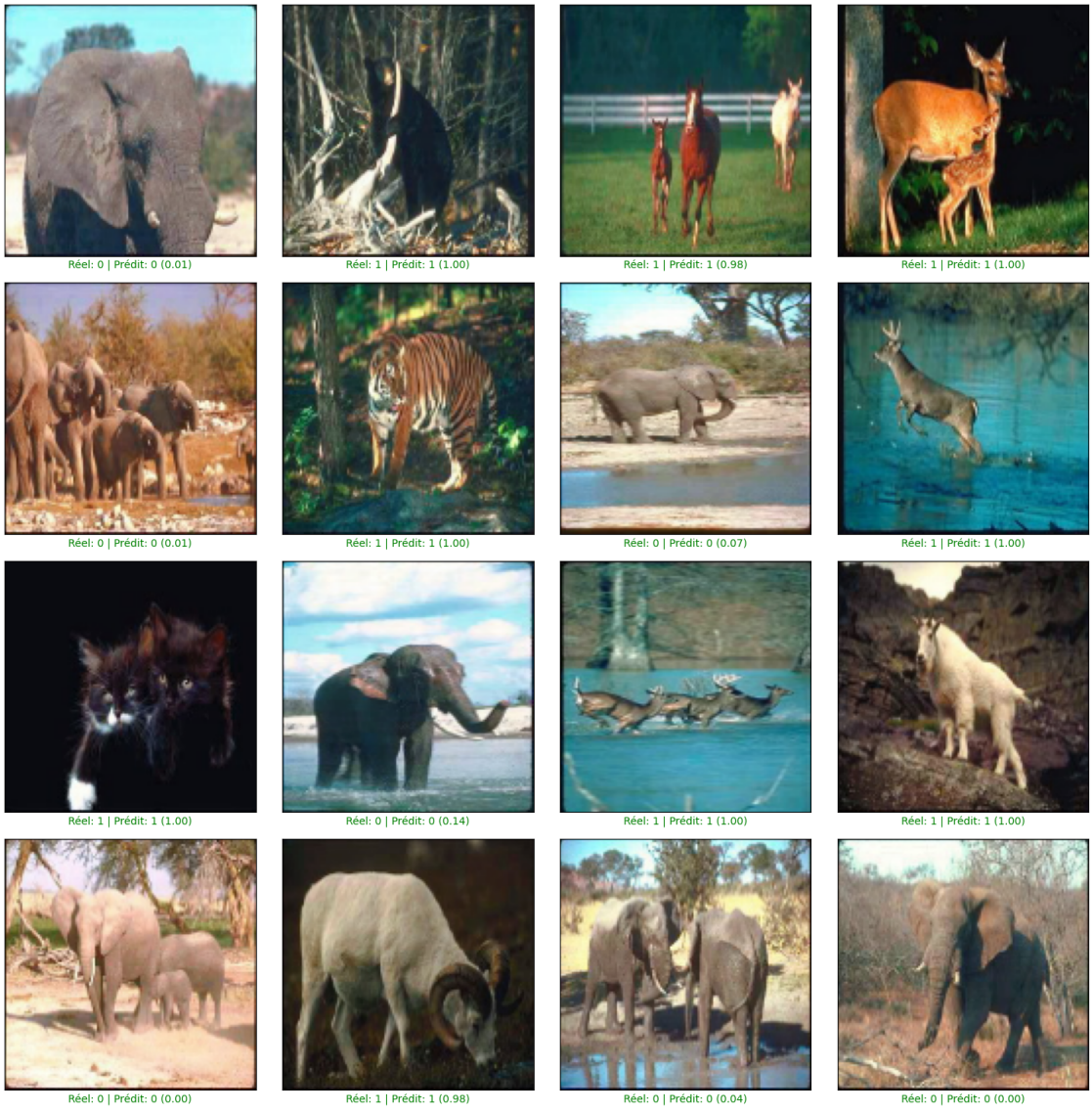


[25]:
```python
# Évaluer sur le test
test_loss, test_acc = model_baseline.evaluate(X_test, y_test, verbose=0)
print(f"Résultats Baseline :")
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

```
Résultats Baseline :
Test Loss: 0.1157
Test Accuracy: 0.9500
```

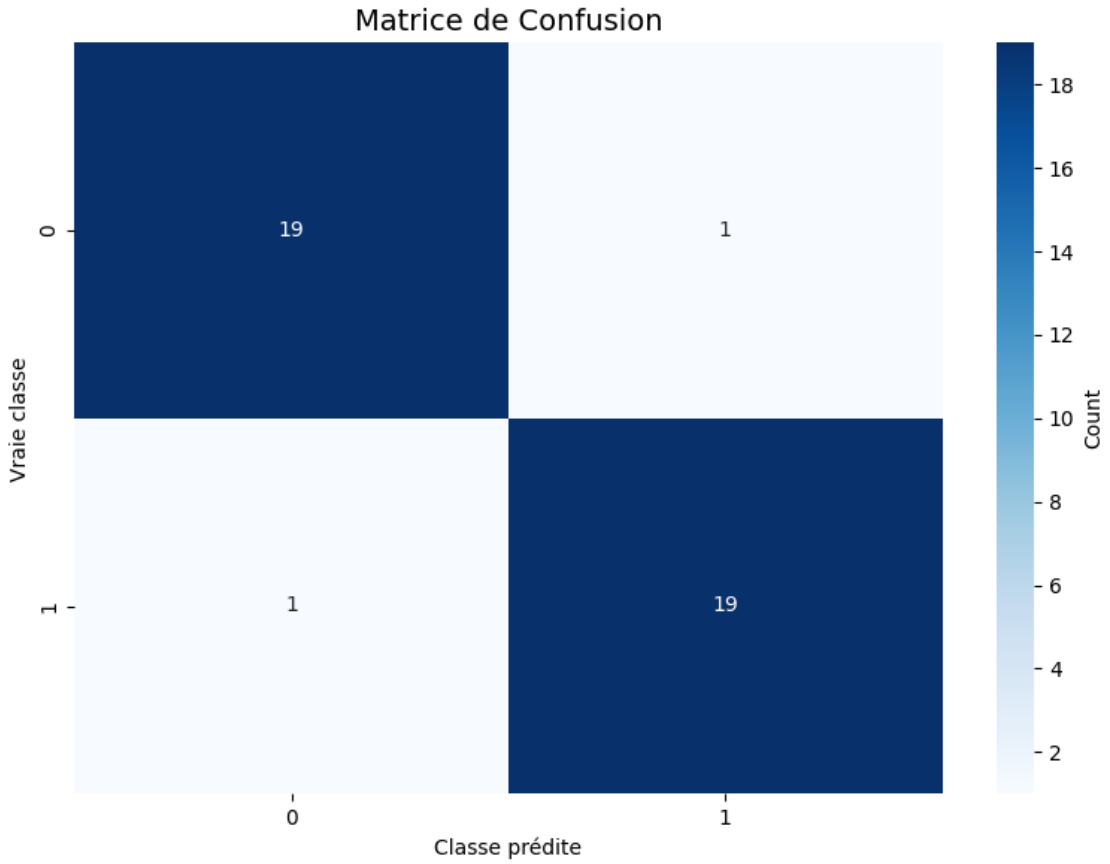
[20]:
```
# Prédictions visuelles
print("\nQuelques prédictions (vert=correct, rouge=erreur):")
plot_predictions(model_baseline, X_test, y_test, n_examples=16)
```

```
Quelques prédictions (vert=correct, rouge=erreur):
```



[21]:
```
# Matrice de confusion
y_pred_baseline = (model_baseline.predict(X_test, verbose=0) >= 0.5).
  ↪astype(int).ravel()
plot_confusion_matrix(y_test, y_pred_baseline)
```

```python
# Rapport détaillé
print("\n" + classification_report(y_test, y_pred_baseline,
                                   target_names=['Classe 0', 'Classe 1']))
```




```
              precision    recall  f1-score   support

    Classe 0       0.95      0.95      0.95        20
    Classe 1       0.95      0.95      0.95        20

    accuracy                           0.95        40
   macro avg       0.95      0.95      0.95        40
weighted avg       0.95      0.95      0.95        40
```

---

3 VARIATION 1 : Plus d'epochs

[26]:
```
# Nouveau modèle (poids réinitialisés)
model_v1 = create_baseline_model(input_shape=(IMG_SIZE, IMG_SIZE, 3))
model_v1.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Entraîner avec plus d'epochs
history_v1 = model_v1.fit(
    X_train, y_train,
    epochs=60,  # Plus d'epochs
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/60
4/4             3s 488ms/step -
accuracy: 0.4297 - loss: 2.5487 - val_accuracy: 0.6250 - val_loss: 1.4916
Epoch 2/60
4/4             1s 340ms/step -
accuracy: 0.4844 - loss: 1.3177 - val_accuracy: 0.4688 - val_loss: 1.7033
Epoch 3/60
4/4             1s 325ms/step -
accuracy: 0.6797 - loss: 0.7141 - val_accuracy: 0.6562 - val_loss: 0.5879
Epoch 4/60
4/4             1s 323ms/step -
accuracy: 0.7344 - loss: 0.5135 - val_accuracy: 0.5938 - val_loss: 0.7885
Epoch 5/60
4/4             1s 332ms/step -
accuracy: 0.7734 - loss: 0.4464 - val_accuracy: 0.8438 - val_loss: 0.3510
Epoch 6/60
4/4             1s 343ms/step -
accuracy: 0.8516 - loss: 0.3161 - val_accuracy: 0.9062 - val_loss: 0.2878
Epoch 7/60
4/4             1s 324ms/step -
accuracy: 0.8984 - loss: 0.2459 - val_accuracy: 0.6875 - val_loss: 0.4924
Epoch 8/60
4/4             1s 347ms/step -
accuracy: 0.8984 - loss: 0.2267 - val_accuracy: 0.8438 - val_loss: 0.2906
Epoch 9/60
4/4             2s 530ms/step -
accuracy: 0.9609 - loss: 0.1851 - val_accuracy: 0.8438 - val_loss: 0.2783
Epoch 10/60
```

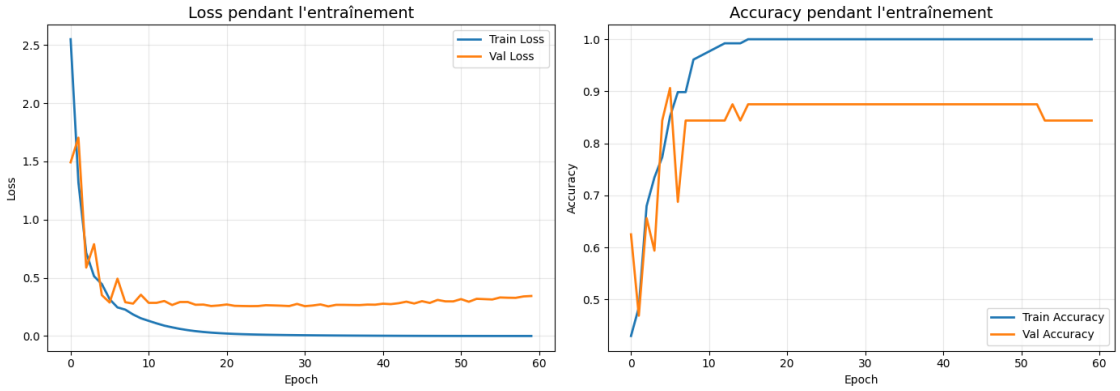
```
4/4            2s 458ms/step -
accuracy: 0.9688 - loss: 0.1524 - val_accuracy: 0.8438 - val_loss: 0.3544
Epoch 11/60
4/4            2s 328ms/step -
accuracy: 0.9766 - loss: 0.1303 - val_accuracy: 0.8438 - val_loss: 0.2849
Epoch 12/60
4/4            3s 338ms/step -
accuracy: 0.9844 - loss: 0.1088 - val_accuracy: 0.8438 - val_loss: 0.2850
Epoch 13/60
4/4            1s 323ms/step -
accuracy: 0.9922 - loss: 0.0892 - val_accuracy: 0.8438 - val_loss: 0.3007
Epoch 14/60
4/4            2s 448ms/step -
accuracy: 0.9922 - loss: 0.0747 - val_accuracy: 0.8750 - val_loss: 0.2666
Epoch 15/60
4/4            2s 340ms/step -
accuracy: 0.9922 - loss: 0.0613 - val_accuracy: 0.8438 - val_loss: 0.2920
Epoch 16/60
4/4            2s 533ms/step -
accuracy: 1.0000 - loss: 0.0506 - val_accuracy: 0.8750 - val_loss: 0.2923
Epoch 17/60
4/4            2s 463ms/step -
accuracy: 1.0000 - loss: 0.0418 - val_accuracy: 0.8750 - val_loss: 0.2686
Epoch 18/60
4/4            2s 334ms/step -
accuracy: 1.0000 - loss: 0.0348 - val_accuracy: 0.8750 - val_loss: 0.2701
Epoch 19/60
4/4            1s 344ms/step -
accuracy: 1.0000 - loss: 0.0293 - val_accuracy: 0.8750 - val_loss: 0.2577
Epoch 20/60
4/4            1s 339ms/step -
accuracy: 1.0000 - loss: 0.0249 - val_accuracy: 0.8750 - val_loss: 0.2626
Epoch 21/60
4/4            1s 348ms/step -
accuracy: 1.0000 - loss: 0.0212 - val_accuracy: 0.8750 - val_loss: 0.2710
Epoch 22/60
4/4            1s 340ms/step -
accuracy: 1.0000 - loss: 0.0183 - val_accuracy: 0.8750 - val_loss: 0.2594
Epoch 23/60
4/4            1s 330ms/step -
accuracy: 1.0000 - loss: 0.0159 - val_accuracy: 0.8750 - val_loss: 0.2578
Epoch 24/60
4/4            1s 363ms/step -
accuracy: 1.0000 - loss: 0.0141 - val_accuracy: 0.8750 - val_loss: 0.2567
Epoch 25/60
4/4            2s 561ms/step -
accuracy: 1.0000 - loss: 0.0125 - val_accuracy: 0.8750 - val_loss: 0.2574
Epoch 26/60
```

```
4/4            2s 378ms/step -
accuracy: 1.0000 - loss: 0.0111 - val_accuracy: 0.8750 - val_loss: 0.2646
Epoch 27/60
4/4            1s 336ms/step -
accuracy: 1.0000 - loss: 0.0100 - val_accuracy: 0.8750 - val_loss: 0.2628
Epoch 28/60
4/4            1s 336ms/step -
accuracy: 1.0000 - loss: 0.0091 - val_accuracy: 0.8750 - val_loss: 0.2605
Epoch 29/60
4/4            1s 343ms/step -
accuracy: 1.0000 - loss: 0.0081 - val_accuracy: 0.8750 - val_loss: 0.2575
Epoch 30/60
4/4            1s 338ms/step -
accuracy: 1.0000 - loss: 0.0074 - val_accuracy: 0.8750 - val_loss: 0.2756
Epoch 31/60
4/4            3s 355ms/step -
accuracy: 1.0000 - loss: 0.0067 - val_accuracy: 0.8750 - val_loss: 0.2564
Epoch 32/60
4/4            1s 371ms/step -
accuracy: 1.0000 - loss: 0.0060 - val_accuracy: 0.8750 - val_loss: 0.2620
Epoch 33/60
4/4            3s 404ms/step -
accuracy: 1.0000 - loss: 0.0054 - val_accuracy: 0.8750 - val_loss: 0.2714
Epoch 34/60
4/4            1s 353ms/step -
accuracy: 1.0000 - loss: 0.0049 - val_accuracy: 0.8750 - val_loss: 0.2546
Epoch 35/60
4/4            3s 369ms/step -
accuracy: 1.0000 - loss: 0.0044 - val_accuracy: 0.8750 - val_loss: 0.2679
Epoch 36/60
4/4            1s 349ms/step -
accuracy: 1.0000 - loss: 0.0040 - val_accuracy: 0.8750 - val_loss: 0.2677
Epoch 37/60
4/4            1s 348ms/step -
accuracy: 1.0000 - loss: 0.0036 - val_accuracy: 0.8750 - val_loss: 0.2666
Epoch 38/60
4/4            1s 348ms/step -
accuracy: 1.0000 - loss: 0.0033 - val_accuracy: 0.8750 - val_loss: 0.2657
Epoch 39/60
4/4            1s 362ms/step -
accuracy: 1.0000 - loss: 0.0030 - val_accuracy: 0.8750 - val_loss: 0.2701
Epoch 40/60
4/4            2s 631ms/step -
accuracy: 1.0000 - loss: 0.0026 - val_accuracy: 0.8750 - val_loss: 0.2692
Epoch 41/60
4/4            1s 334ms/step -
accuracy: 1.0000 - loss: 0.0024 - val_accuracy: 0.8750 - val_loss: 0.2775
Epoch 42/60
```

```
4/4            2s 335ms/step -
accuracy: 1.0000 - loss: 0.0021 - val_accuracy: 0.8750 - val_loss: 0.2739
Epoch 43/60
4/4            1s 340ms/step -
accuracy: 1.0000 - loss: 0.0018 - val_accuracy: 0.8750 - val_loss: 0.2817
Epoch 44/60
4/4            3s 355ms/step -
accuracy: 1.0000 - loss: 0.0016 - val_accuracy: 0.8750 - val_loss: 0.2951
Epoch 45/60
4/4            3s 349ms/step -
accuracy: 1.0000 - loss: 0.0014 - val_accuracy: 0.8750 - val_loss: 0.2794
Epoch 46/60
4/4            2s 592ms/step -
accuracy: 1.0000 - loss: 0.0012 - val_accuracy: 0.8750 - val_loss: 0.2991
Epoch 47/60
4/4            2s 413ms/step -
accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.8750 - val_loss: 0.2843
Epoch 48/60
4/4            1s 340ms/step -
accuracy: 1.0000 - loss: 9.3968e-04 - val_accuracy: 0.8750 - val_loss: 0.3100
Epoch 49/60
4/4            1s 340ms/step -
accuracy: 1.0000 - loss: 8.3241e-04 - val_accuracy: 0.8750 - val_loss: 0.2976
Epoch 50/60
4/4            1s 351ms/step -
accuracy: 1.0000 - loss: 7.3447e-04 - val_accuracy: 0.8750 - val_loss: 0.2973
Epoch 51/60
4/4            3s 350ms/step -
accuracy: 1.0000 - loss: 6.4310e-04 - val_accuracy: 0.8750 - val_loss: 0.3170
Epoch 52/60
4/4            3s 357ms/step -
accuracy: 1.0000 - loss: 5.8239e-04 - val_accuracy: 0.8750 - val_loss: 0.2942
Epoch 53/60
4/4            3s 486ms/step -
accuracy: 1.0000 - loss: 5.1478e-04 - val_accuracy: 0.8750 - val_loss: 0.3196
Epoch 54/60
4/4            1s 338ms/step -
accuracy: 1.0000 - loss: 4.6810e-04 - val_accuracy: 0.8438 - val_loss: 0.3168
Epoch 55/60
4/4            1s 326ms/step -
accuracy: 1.0000 - loss: 4.2347e-04 - val_accuracy: 0.8438 - val_loss: 0.3138
Epoch 56/60
4/4            1s 333ms/step -
accuracy: 1.0000 - loss: 3.8309e-04 - val_accuracy: 0.8438 - val_loss: 0.3310
Epoch 57/60
4/4            1s 345ms/step -
accuracy: 1.0000 - loss: 3.5212e-04 - val_accuracy: 0.8438 - val_loss: 0.3288
Epoch 58/60
```

```
4/4             1s 343ms/step -
accuracy: 1.0000 - loss: 3.2325e-04 - val_accuracy: 0.8438 - val_loss: 0.3279
Epoch 59/60
4/4             1s 339ms/step -
accuracy: 1.0000 - loss: 2.9691e-04 - val_accuracy: 0.8438 - val_loss: 0.3401
Epoch 60/60
4/4             1s 349ms/step -
accuracy: 1.0000 - loss: 2.7510e-04 - val_accuracy: 0.8438 - val_loss: 0.3439
```

[28]:
```python
# Courbes
plot_history(history_v1)

# Évaluation
test_loss_v1, test_acc_v1 = model_v1.evaluate(X_test, y_test, verbose=0)
print(f"\n Variation 1 (60 epochs)")
print(f"Test Accuracy: {test_acc_v1:.4f}")
print(f"Comparaison Baseline: {test_acc:.4f} → V1: {test_acc_v1:.4f}")
```




```
 Variation 1 (60 epochs)
Test Accuracy: 0.9250
Comparaison Baseline: 0.9500 → V1: 0.9250
```

---

4 VARIATION 2 : Learning rate différent

[29]:
```python
# Nouveau modèle avec learning rate plus petit
model_v2 = create_baseline_model(input_shape=(IMG_SIZE, IMG_SIZE, 3))
model_v2.compile(
    optimizer=Adam(learning_rate=1e-4),  # 10x plus petit
    loss='binary_crossentropy',
    metrics=['accuracy']
```

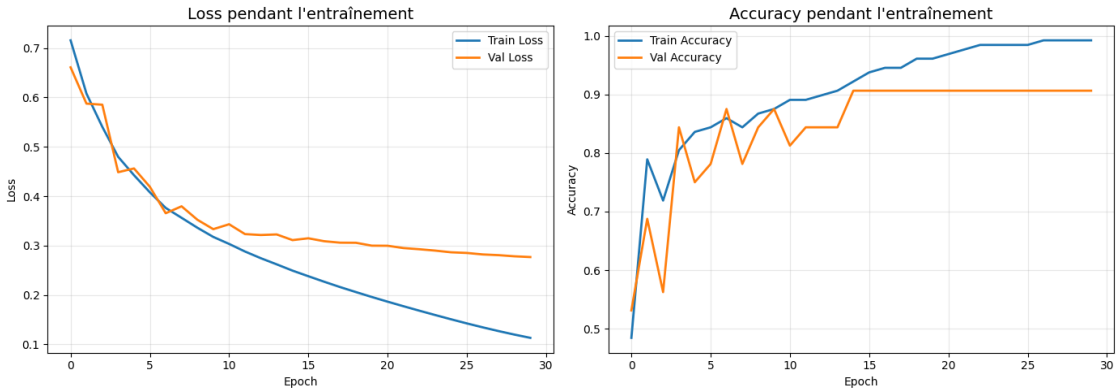
```
)

history_v2 = model_v2.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/30
4/4             2s 399ms/step -
accuracy: 0.4844 - loss: 0.7157 - val_accuracy: 0.5312 - val_loss: 0.6610
Epoch 2/30
4/4             1s 346ms/step -
accuracy: 0.7891 - loss: 0.6079 - val_accuracy: 0.6875 - val_loss: 0.5875
Epoch 3/30
4/4             3s 344ms/step -
accuracy: 0.7188 - loss: 0.5405 - val_accuracy: 0.5625 - val_loss: 0.5854
Epoch 4/30
4/4             1s 338ms/step -
accuracy: 0.8047 - loss: 0.4798 - val_accuracy: 0.8438 - val_loss: 0.4486
Epoch 5/30
4/4             2s 604ms/step -
accuracy: 0.8359 - loss: 0.4426 - val_accuracy: 0.7500 - val_loss: 0.4562
Epoch 6/30
4/4             2s 405ms/step -
accuracy: 0.8438 - loss: 0.4079 - val_accuracy: 0.7812 - val_loss: 0.4191
Epoch 7/30
4/4             1s 335ms/step -
accuracy: 0.8594 - loss: 0.3763 - val_accuracy: 0.8750 - val_loss: 0.3655
Epoch 8/30
4/4             3s 349ms/step -
accuracy: 0.8438 - loss: 0.3563 - val_accuracy: 0.7812 - val_loss: 0.3796
Epoch 9/30
4/4             1s 354ms/step -
accuracy: 0.8672 - loss: 0.3361 - val_accuracy: 0.8438 - val_loss: 0.3523
Epoch 10/30
4/4             1s 342ms/step -
accuracy: 0.8750 - loss: 0.3176 - val_accuracy: 0.8750 - val_loss: 0.3333
Epoch 11/30
4/4             1s 333ms/step -
accuracy: 0.8906 - loss: 0.3034 - val_accuracy: 0.8125 - val_loss: 0.3432
Epoch 12/30
4/4             3s 562ms/step -
accuracy: 0.8906 - loss: 0.2880 - val_accuracy: 0.8438 - val_loss: 0.3233
Epoch 13/30
```

```
4/4            2s 447ms/step -
accuracy: 0.8984 - loss: 0.2744 - val_accuracy: 0.8438 - val_loss: 0.3214
Epoch 14/30
4/4            1s 342ms/step -
accuracy: 0.9062 - loss: 0.2620 - val_accuracy: 0.8438 - val_loss: 0.3226
Epoch 15/30
4/4            1s 338ms/step -
accuracy: 0.9219 - loss: 0.2493 - val_accuracy: 0.9062 - val_loss: 0.3111
Epoch 16/30
4/4            1s 342ms/step -
accuracy: 0.9375 - loss: 0.2379 - val_accuracy: 0.9062 - val_loss: 0.3148
Epoch 17/30
4/4            1s 333ms/step -
accuracy: 0.9453 - loss: 0.2268 - val_accuracy: 0.9062 - val_loss: 0.3088
Epoch 18/30
4/4            1s 327ms/step -
accuracy: 0.9453 - loss: 0.2160 - val_accuracy: 0.9062 - val_loss: 0.3059
Epoch 19/30
4/4            3s 332ms/step -
accuracy: 0.9609 - loss: 0.2059 - val_accuracy: 0.9062 - val_loss: 0.3057
Epoch 20/30
4/4            2s 561ms/step -
accuracy: 0.9609 - loss: 0.1959 - val_accuracy: 0.9062 - val_loss: 0.2999
Epoch 21/30
4/4            2s 464ms/step -
accuracy: 0.9688 - loss: 0.1864 - val_accuracy: 0.9062 - val_loss: 0.2997
Epoch 22/30
4/4            1s 346ms/step -
accuracy: 0.9766 - loss: 0.1772 - val_accuracy: 0.9062 - val_loss: 0.2950
Epoch 23/30
4/4            1s 348ms/step -
accuracy: 0.9844 - loss: 0.1682 - val_accuracy: 0.9062 - val_loss: 0.2926
Epoch 24/30
4/4            1s 334ms/step -
accuracy: 0.9844 - loss: 0.1594 - val_accuracy: 0.9062 - val_loss: 0.2898
Epoch 25/30
4/4            1s 343ms/step -
accuracy: 0.9844 - loss: 0.1507 - val_accuracy: 0.9062 - val_loss: 0.2864
Epoch 26/30
4/4            1s 336ms/step -
accuracy: 0.9844 - loss: 0.1424 - val_accuracy: 0.9062 - val_loss: 0.2851
Epoch 27/30
4/4            1s 361ms/step -
accuracy: 0.9922 - loss: 0.1344 - val_accuracy: 0.9062 - val_loss: 0.2821
Epoch 28/30
4/4            1s 349ms/step -
accuracy: 0.9922 - loss: 0.1268 - val_accuracy: 0.9062 - val_loss: 0.2806
Epoch 29/30
```

```
4/4             2s 570ms/step -
accuracy: 0.9922 - loss: 0.1197 - val_accuracy: 0.9062 - val_loss: 0.2783
Epoch 30/30
4/4             2s 335ms/step -
accuracy: 0.9922 - loss: 0.1131 - val_accuracy: 0.9062 - val_loss: 0.2768
```

[30]:
```python
plot_history(history_v2)

test_loss_v2, test_acc_v2 = model_v2.evaluate(X_test, y_test, verbose=0)
print(f"\n Variation 2 (LR=1e-4)")
print(f"Test Accuracy: {test_acc_v2:.4f}")
```




```
 Variation 2 (LR=1e-4)
Test Accuracy: 0.9500
```

---

5 VARIATION 3 : Batch size différent

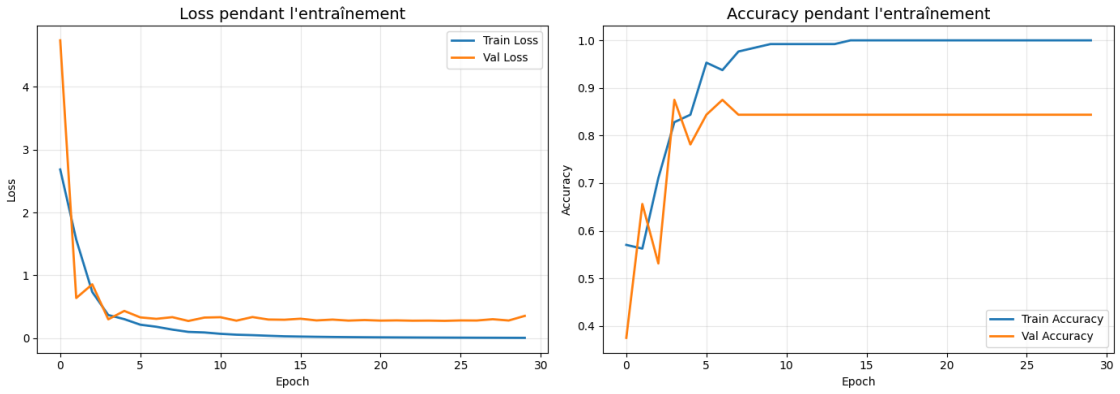
[31]:
```python
# Batch size plus petit
model_v3 = create_baseline_model(input_shape=(IMG_SIZE, IMG_SIZE, 3))
model_v3.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_v3 = model_v3.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,  # Plus petit batch
    validation_split=0.2,
```

```
    verbose=1
)
```

```
Epoch 1/30
8/8             2s 185ms/step -
accuracy: 0.5703 - loss: 2.6834 - val_accuracy: 0.3750 - val_loss: 4.7372
Epoch 2/30
8/8             1s 145ms/step -
accuracy: 0.5625 - loss: 1.5674 - val_accuracy: 0.6562 - val_loss: 0.6387
Epoch 3/30
8/8             1s 161ms/step -
accuracy: 0.7109 - loss: 0.7350 - val_accuracy: 0.5312 - val_loss: 0.8568
Epoch 4/30
8/8             2s 269ms/step -
accuracy: 0.8281 - loss: 0.3679 - val_accuracy: 0.8750 - val_loss: 0.2999
Epoch 5/30
8/8             2s 208ms/step -
accuracy: 0.8438 - loss: 0.3026 - val_accuracy: 0.7812 - val_loss: 0.4335
Epoch 6/30
8/8             2s 150ms/step -
accuracy: 0.9531 - loss: 0.2144 - val_accuracy: 0.8438 - val_loss: 0.3305
Epoch 7/30
8/8             1s 148ms/step -
accuracy: 0.9375 - loss: 0.1817 - val_accuracy: 0.8750 - val_loss: 0.3076
Epoch 8/30
8/8             1s 146ms/step -
accuracy: 0.9766 - loss: 0.1366 - val_accuracy: 0.8438 - val_loss: 0.3342
Epoch 9/30
8/8             1s 162ms/step -
accuracy: 0.9844 - loss: 0.1000 - val_accuracy: 0.8438 - val_loss: 0.2738
Epoch 10/30
8/8             1s 149ms/step -
accuracy: 0.9922 - loss: 0.0908 - val_accuracy: 0.8438 - val_loss: 0.3278
Epoch 11/30
8/8             1s 153ms/step -
accuracy: 0.9922 - loss: 0.0697 - val_accuracy: 0.8438 - val_loss: 0.3333
Epoch 12/30
8/8             1s 154ms/step -
accuracy: 0.9922 - loss: 0.0554 - val_accuracy: 0.8438 - val_loss: 0.2786
Epoch 13/30
8/8             2s 270ms/step -
accuracy: 0.9922 - loss: 0.0481 - val_accuracy: 0.8438 - val_loss: 0.3360
Epoch 14/30
8/8             2s 152ms/step -
accuracy: 0.9922 - loss: 0.0385 - val_accuracy: 0.8438 - val_loss: 0.2964
Epoch 15/30
8/8             1s 151ms/step -
```

```
accuracy: 1.0000 - loss: 0.0297 - val_accuracy: 0.8438 - val_loss: 0.2932
Epoch 16/30
8/8            1s 148ms/step -
accuracy: 1.0000 - loss: 0.0255 - val_accuracy: 0.8438 - val_loss: 0.3097
Epoch 17/30
8/8            1s 149ms/step -
accuracy: 1.0000 - loss: 0.0218 - val_accuracy: 0.8438 - val_loss: 0.2825
Epoch 18/30
8/8            1s 152ms/step -
accuracy: 1.0000 - loss: 0.0189 - val_accuracy: 0.8438 - val_loss: 0.2956
Epoch 19/30
8/8            1s 146ms/step -
accuracy: 1.0000 - loss: 0.0167 - val_accuracy: 0.8438 - val_loss: 0.2797
Epoch 20/30
8/8            1s 154ms/step -
accuracy: 1.0000 - loss: 0.0148 - val_accuracy: 0.8438 - val_loss: 0.2882
Epoch 21/30
8/8            1s 146ms/step -
accuracy: 1.0000 - loss: 0.0133 - val_accuracy: 0.8438 - val_loss: 0.2795
Epoch 22/30
8/8            2s 214ms/step -
accuracy: 1.0000 - loss: 0.0120 - val_accuracy: 0.8438 - val_loss: 0.2830
Epoch 23/30
8/8            2s 235ms/step -
accuracy: 1.0000 - loss: 0.0109 - val_accuracy: 0.8438 - val_loss: 0.2781
Epoch 24/30
8/8            1s 154ms/step -
accuracy: 1.0000 - loss: 0.0099 - val_accuracy: 0.8438 - val_loss: 0.2796
Epoch 25/30
8/8            1s 156ms/step -
accuracy: 1.0000 - loss: 0.0091 - val_accuracy: 0.8438 - val_loss: 0.2759
Epoch 26/30
8/8            1s 155ms/step -
accuracy: 1.0000 - loss: 0.0083 - val_accuracy: 0.8438 - val_loss: 0.2823
Epoch 27/30
8/8            1s 154ms/step -
accuracy: 1.0000 - loss: 0.0073 - val_accuracy: 0.8438 - val_loss: 0.2809
Epoch 28/30
8/8            1s 150ms/step -
accuracy: 1.0000 - loss: 0.0067 - val_accuracy: 0.8438 - val_loss: 0.3014
Epoch 29/30
8/8            1s 153ms/step -
accuracy: 1.0000 - loss: 0.0059 - val_accuracy: 0.8438 - val_loss: 0.2811
Epoch 30/30
8/8            1s 152ms/step -
accuracy: 1.0000 - loss: 0.0052 - val_accuracy: 0.8438 - val_loss: 0.3545
```

[32]:
```
plot_history(history_v3)

test_loss_v3, test_acc_v3 = model_v3.evaluate(X_test, y_test, verbose=0)
print(f"\n Variation 3 (batch_size=16)")
print(f"Test Accuracy: {test_acc_v3:.4f}")
```




```
 Variation 3 (batch_size=16)
Test Accuracy: 0.9500
```

---

6 MODÈLE AMÉLIORÉ : Plus de couches + Dropout + BatchNorm

[33]:
```
def create_improved_model(input_shape=(128, 128, 3)):
    """
    CNN amélioré avec plus de couches et régularisation.
    """
    inputs = Input(shape=input_shape)

    # Bloc 1
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.25)(x)

    # Bloc 2
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.25)(x)
```

```
    # Bloc 3
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.25)(x)

    # Classification
    x = Flatten()(x)
    x = Dense(128, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.5)(x)
    outputs = Dense(1, activation='sigmoid')(x)

    model = Model(inputs, outputs, name='CNN_Improved')
    return model
```

[34]:
```
# Créer et compiler
model_improved = create_improved_model(input_shape=(IMG_SIZE, IMG_SIZE, 3))
model_improved.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model_improved.summary()
```

```
Model: "CNN_Improved"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 128, 128, 32)	896
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
dropout (Dropout)	(None, 64, 64, 32)	0
conv2d_1 (Conv2D)	(None, 64, 64, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 64, 64, 64)	256

max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
dropout_1 (Dropout)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 32, 32, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 32, 32, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0
dropout_2 (Dropout)	(None, 16, 16, 128)	0
flatten (Flatten)	(None, 32768)	0
dense (Dense)	(None, 128)	4,194,432
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 4,289,217 (16.36 MB)

Trainable params: 4,288,513 (16.36 MB)

Non-trainable params: 704 (2.75 KB)

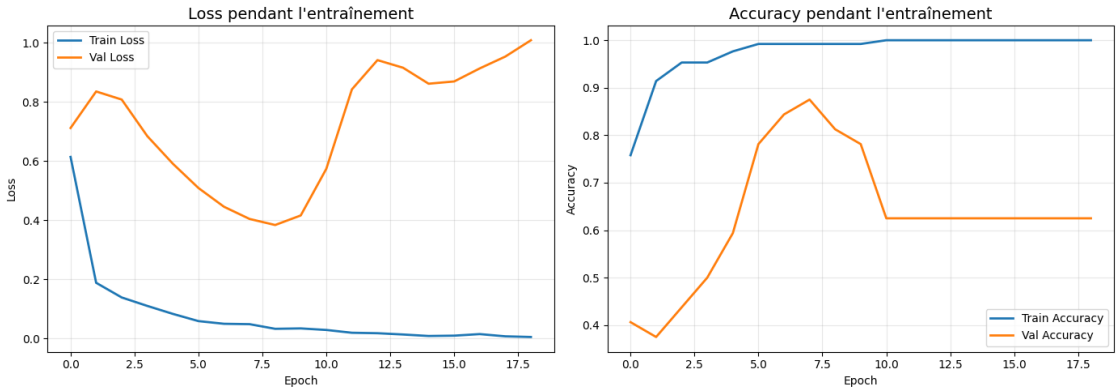
[35]:
```
# EarlyStopping pour éviter le surapprentissage
es = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

# Entraîner
history_improved = model_improved.fit(
    X_train, y_train,
    epochs=100,  # On peut mettre beaucoup, EarlyStopping va arrêter
    batch_size=32,
    validation_split=0.2,
```

```
    callbacks=[es],
    verbose=1
)
```

```
Epoch 1/100
4/4             12s 2s/step -
accuracy: 0.7578 - loss: 0.6135 - val_accuracy: 0.4062 - val_loss: 0.7113
Epoch 2/100
4/4             11s 2s/step -
accuracy: 0.9141 - loss: 0.1879 - val_accuracy: 0.3750 - val_loss: 0.8349
Epoch 3/100
4/4             10s 2s/step -
accuracy: 0.9531 - loss: 0.1382 - val_accuracy: 0.4375 - val_loss: 0.8075
Epoch 4/100
4/4             9s 2s/step -
accuracy: 0.9531 - loss: 0.1097 - val_accuracy: 0.5000 - val_loss: 0.6839
Epoch 5/100
4/4             10s 2s/step -
accuracy: 0.9766 - loss: 0.0827 - val_accuracy: 0.5938 - val_loss: 0.5905
Epoch 6/100
4/4             10s 2s/step -
accuracy: 0.9922 - loss: 0.0583 - val_accuracy: 0.7812 - val_loss: 0.5084
Epoch 7/100
4/4             8s 2s/step -
accuracy: 0.9922 - loss: 0.0492 - val_accuracy: 0.8438 - val_loss: 0.4450
Epoch 8/100
4/4             7s 2s/step -
accuracy: 0.9922 - loss: 0.0479 - val_accuracy: 0.8750 - val_loss: 0.4037
Epoch 9/100
4/4             8s 2s/step -
accuracy: 0.9922 - loss: 0.0322 - val_accuracy: 0.8125 - val_loss: 0.3833
Epoch 10/100
4/4             7s 2s/step -
accuracy: 0.9922 - loss: 0.0336 - val_accuracy: 0.7812 - val_loss: 0.4156
Epoch 11/100
4/4             10s 2s/step -
accuracy: 1.0000 - loss: 0.0284 - val_accuracy: 0.6250 - val_loss: 0.5734
Epoch 12/100
4/4             8s 2s/step -
accuracy: 1.0000 - loss: 0.0189 - val_accuracy: 0.6250 - val_loss: 0.8419
Epoch 13/100
4/4             10s 2s/step -
accuracy: 1.0000 - loss: 0.0174 - val_accuracy: 0.6250 - val_loss: 0.9409
Epoch 14/100
4/4             7s 2s/step -
accuracy: 1.0000 - loss: 0.0130 - val_accuracy: 0.6250 - val_loss: 0.9152
Epoch 15/100
```

```
4/4             8s 2s/step -
accuracy: 1.0000 - loss: 0.0080 - val_accuracy: 0.6250 - val_loss: 0.8611
Epoch 16/100
4/4             7s 2s/step -
accuracy: 1.0000 - loss: 0.0091 - val_accuracy: 0.6250 - val_loss: 0.8688
Epoch 17/100
4/4             8s 2s/step -
accuracy: 1.0000 - loss: 0.0144 - val_accuracy: 0.6250 - val_loss: 0.9129
Epoch 18/100
4/4             8s 2s/step -
accuracy: 1.0000 - loss: 0.0068 - val_accuracy: 0.6250 - val_loss: 0.9530
Epoch 19/100
4/4             9s 2s/step -
accuracy: 1.0000 - loss: 0.0046 - val_accuracy: 0.6250 - val_loss: 1.0081
Epoch 19: early stopping
Restoring model weights from the end of the best epoch: 9.
```

[37]:
```python
# Courbes
plot_history(history_improved)

# Évaluation
test_loss_improved, test_acc_improved = model_improved.evaluate(X_test, y_test,␣
 ↪verbose=0)
print(f"\nModèle Amélioré")
print(f"Test Accuracy: {test_acc_improved:.4f}")
```




```
Modèle Amélioré
Test Accuracy: 0.8500
```

6.1 Analyse : Overfitting sévère détecté

Observation des courbes :

6.1.1 Graphique Loss

- **Train Loss** : Descend à ~0.01 (quasi parfait)
- **Val Loss** : Remonte à 1.0 après epoch 7
- **Diagnostic** : OVERFITTING MASSIF

6.1.2 Graphique Accuracy

- **Train Accuracy** : 100% (le modèle mémorise parfaitement le train)
- **Val Accuracy** : Plafonne à 63% puis stagne
- **Diagnostic** : Le modèle échoue sur les données de validation

Test Accuracy : 85% - Résultat trompeur car il y a eu de la chance

6.1.3 Conclusion

Ce modèle est **trop complexe** pour notre petit dataset (200 images) : - 3 blocs de convolution - BatchNormalization partout - Dropout 50% - **Résultat** : Le modèle mémorise au lieu d'apprendre des patterns généraux

Leçon apprise : Un modèle complexe + petit dataset = Overfitting garanti

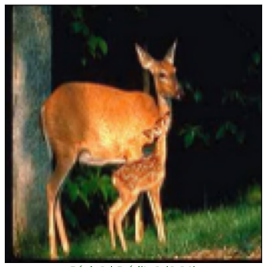
Prochaine étape : Simplifier l'architecture pour mieux généraliser.

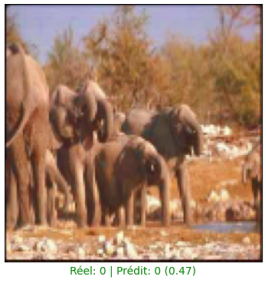
[38]:
```
# Prédictions
plot_predictions(model_improved, X_test, y_test, n_examples=16)

# Matrice de confusion
y_pred_improved = (model_improved.predict(X_test, verbose=0) >= 0.5).
  ↪astype(int).ravel()
plot_confusion_matrix(y_test, y_pred_improved)
```


Réel: 0 | Prédit: 0 (0.37)

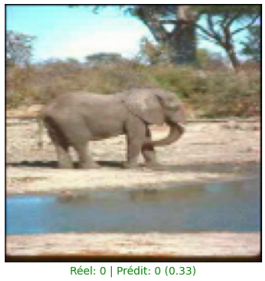

Réel: 1 | Prédit: 1 (0.78)


Réel: 1 | Prédit: 1 (0.94)

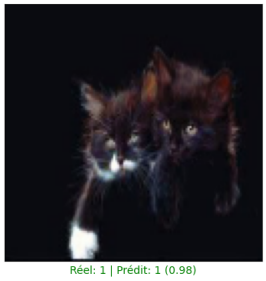

Réel: 1 | Prédit: 1 (0.94)


Réel: 0 | Prédit: 0 (0.47)


Réel: 1 | Prédit: 1 (0.90)


Réel: 0 | Prédit: 0 (0.33)


Réel: 1 | Prédit: 1 (0.92)

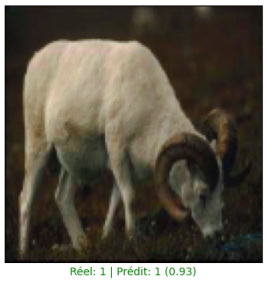

Réel: 1 | Prédit: 1 (0.98)

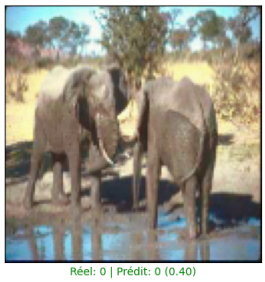

Réel: 0 | Prédit: 0 (0.35)


Réel: 1 | Prédit: 1 (0.85)

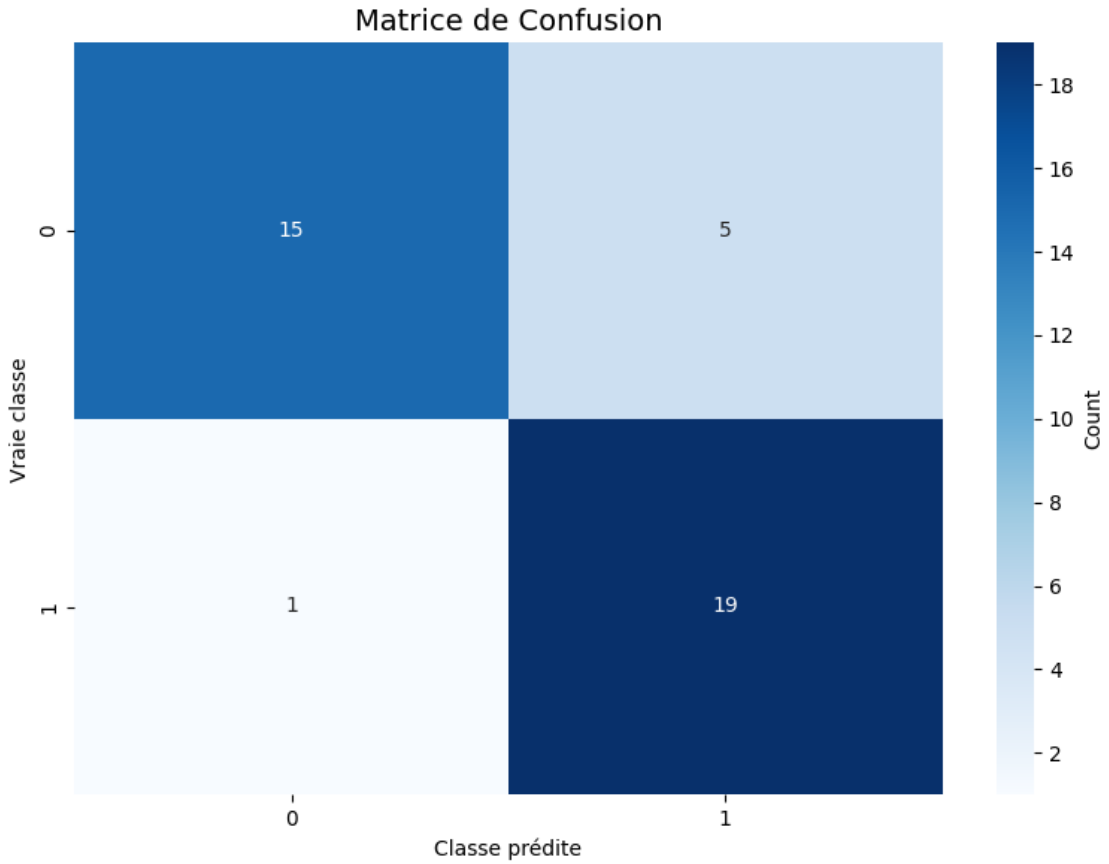

Réel: 1 | Prédit: 1 (0.90)


Réel: 0 | Prédit: 0 (0.30)


Réel: 1 | Prédit: 1 (0.93)


Réel: 0 | Prédit: 0 (0.40)


Réel: 0 | Prédit: 0 (0.32)



7 MODÈLE AMÉLIORÉ V2 : Architecture simplifiée

Après avoir observé l'overfitting du modèle complexe, nous créons une **version allégée** :

Changements par rapport au modèle avec overfitting : - Suppression de BatchNormalization (utile surtout pour grands datasets) - Dropout réduit : 20% sur les blocs Conv (au lieu de 25%) et 40% sur le Dense (au lieu de 50%) - Dense(256) au lieu de Dense(128) — plus de capacité de décision en sortie - 3 blocs Conv conservés mais sans BatchNorm pour rester léger

Objectif : Trouver le bon équilibre entre capacité d'apprentissage et généralisation.

[51]:
```
def create_improved_v2_model(input_shape=(128, 128, 3)):
    """
    CNN amélioré SIMPLIFIÉ - adapté au petit dataset.
    """
    inputs = Input(shape=input_shape)

    # Bloc 1
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    x = MaxPooling2D((2, 2))(x)
```

```
    x = Dropout(0.2)(x)

    # Bloc 2
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.2)(x)

    # Bloc 3 léger (ajouté)
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.25)(x)

    # Classification
    x = Flatten()(x)
    x = Dense(256, activation='relu')(x)  # Plus de neurones
    x = Dropout(0.4)(x)
    outputs = Dense(1, activation='sigmoid')(x)

    model = Model(inputs, outputs, name='CNN_Improved_V2')
    return model

# Créer et compiler
model_v2 = create_improved_v2_model(input_shape=(IMG_SIZE, IMG_SIZE, 3))
model_v2.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model_v2.summary()
```

Model: "CNN_Improved_V2"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 128, 128, 3)	0
conv2d_9 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_9 (MaxPooling2D)	(None, 64, 64, 32)	0
dropout_13 (Dropout)	(None, 64, 64, 32)	0
conv2d_10 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_10 (MaxPooling2D)	(None, 32, 32, 64)	0

```
 dropout_14 (Dropout)             (None, 32, 32, 64)                0

 conv2d_11 (Conv2D)               (None, 32, 32, 128)           73,856

 max_pooling2d_11 (MaxPooling2D)  (None, 16, 16, 128)               0

 dropout_15 (Dropout)             (None, 16, 16, 128)               0

 flatten_4 (Flatten)              (None, 32768)                     0

 dense_8 (Dense)                  (None, 256)                8,388,864

 dropout_16 (Dropout)             (None, 256)                       0

 dense_9 (Dense)                  (None, 1)                       257


 Total params: 8,482,369 (32.36 MB)

 Trainable params: 8,482,369 (32.36 MB)

 Non-trainable params: 0 (0.00 B)
```

[52]:
```python
# EarlyStopping
es_v2 = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

# Entraîner
history_v2 = model_v2.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[es_v2],
    verbose=1
)
```

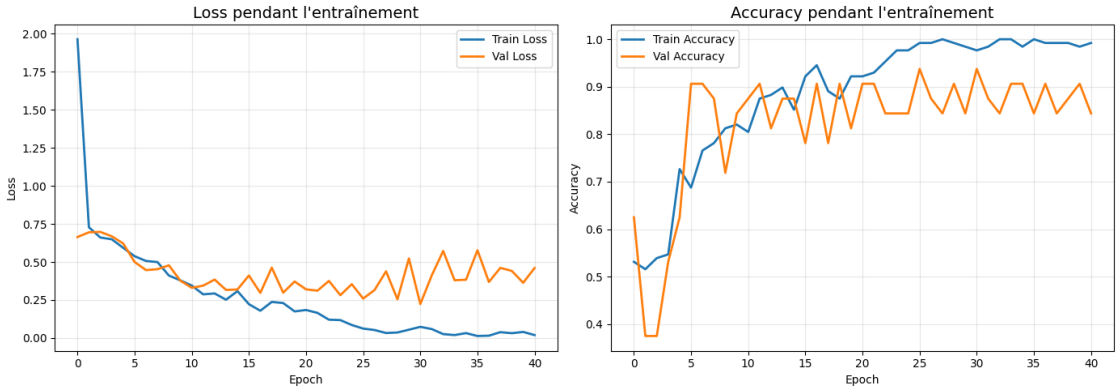
```
Epoch 1/100
4/4             9s 2s/step -
accuracy: 0.5312 - loss: 1.9638 - val_accuracy: 0.6250 - val_loss: 0.6639
```

```
Epoch 2/100
4/4            5s 1s/step -
accuracy: 0.5156 - loss: 0.7280 - val_accuracy: 0.3750 - val_loss: 0.6949
Epoch 3/100
4/4            6s 2s/step -
accuracy: 0.5391 - loss: 0.6606 - val_accuracy: 0.3750 - val_loss: 0.6976
Epoch 4/100
4/4            9s 1s/step -
accuracy: 0.5469 - loss: 0.6490 - val_accuracy: 0.5312 - val_loss: 0.6686
Epoch 5/100
4/4            6s 2s/step -
accuracy: 0.7266 - loss: 0.5930 - val_accuracy: 0.6250 - val_loss: 0.6217
Epoch 6/100
4/4            10s 2s/step -
accuracy: 0.6875 - loss: 0.5378 - val_accuracy: 0.9062 - val_loss: 0.4995
Epoch 7/100
4/4            5s 1s/step -
accuracy: 0.7656 - loss: 0.5065 - val_accuracy: 0.9062 - val_loss: 0.4466
Epoch 8/100
4/4            5s 1s/step -
accuracy: 0.7812 - loss: 0.4994 - val_accuracy: 0.8750 - val_loss: 0.4524
Epoch 9/100
4/4            6s 2s/step -
accuracy: 0.8125 - loss: 0.4107 - val_accuracy: 0.7188 - val_loss: 0.4780
Epoch 10/100
4/4            9s 1s/step -
accuracy: 0.8203 - loss: 0.3790 - val_accuracy: 0.8438 - val_loss: 0.3773
Epoch 11/100
4/4            10s 1s/step -
accuracy: 0.8047 - loss: 0.3450 - val_accuracy: 0.8750 - val_loss: 0.3303
Epoch 12/100
4/4            6s 2s/step -
accuracy: 0.8750 - loss: 0.2873 - val_accuracy: 0.9062 - val_loss: 0.3442
Epoch 13/100
4/4            9s 1s/step -
accuracy: 0.8828 - loss: 0.2931 - val_accuracy: 0.8125 - val_loss: 0.3844
Epoch 14/100
4/4            6s 1s/step -
accuracy: 0.8984 - loss: 0.2518 - val_accuracy: 0.8750 - val_loss: 0.3162
Epoch 15/100
4/4            5s 1s/step -
accuracy: 0.8516 - loss: 0.3084 - val_accuracy: 0.8750 - val_loss: 0.3192
Epoch 16/100
4/4            6s 2s/step -
accuracy: 0.9219 - loss: 0.2227 - val_accuracy: 0.7812 - val_loss: 0.4109
Epoch 17/100
4/4            5s 1s/step -
accuracy: 0.9453 - loss: 0.1794 - val_accuracy: 0.9062 - val_loss: 0.2967
```

```
Epoch 18/100
4/4            5s 1s/step -
accuracy: 0.8906 - loss: 0.2374 - val_accuracy: 0.7812 - val_loss: 0.4628
Epoch 19/100
4/4            6s 1s/step -
accuracy: 0.8750 - loss: 0.2301 - val_accuracy: 0.9062 - val_loss: 0.2982
Epoch 20/100
4/4            10s 2s/step -
accuracy: 0.9219 - loss: 0.1750 - val_accuracy: 0.8125 - val_loss: 0.3715
Epoch 21/100
4/4            9s 1s/step -
accuracy: 0.9219 - loss: 0.1843 - val_accuracy: 0.9062 - val_loss: 0.3201
Epoch 22/100
4/4            6s 2s/step -
accuracy: 0.9297 - loss: 0.1653 - val_accuracy: 0.9062 - val_loss: 0.3113
Epoch 23/100
4/4            5s 1s/step -
accuracy: 0.9531 - loss: 0.1206 - val_accuracy: 0.8438 - val_loss: 0.3754
Epoch 24/100
4/4            6s 2s/step -
accuracy: 0.9766 - loss: 0.1181 - val_accuracy: 0.8438 - val_loss: 0.2817
Epoch 25/100
4/4            5s 1s/step -
accuracy: 0.9766 - loss: 0.0858 - val_accuracy: 0.8438 - val_loss: 0.3541
Epoch 26/100
4/4            5s 1s/step -
accuracy: 0.9922 - loss: 0.0621 - val_accuracy: 0.9375 - val_loss: 0.2595
Epoch 27/100
4/4            6s 1s/step -
accuracy: 0.9922 - loss: 0.0527 - val_accuracy: 0.8750 - val_loss: 0.3160
Epoch 28/100
4/4            5s 1s/step -
accuracy: 1.0000 - loss: 0.0330 - val_accuracy: 0.8438 - val_loss: 0.4384
Epoch 29/100
4/4            7s 2s/step -
accuracy: 0.9922 - loss: 0.0365 - val_accuracy: 0.9062 - val_loss: 0.2546
Epoch 30/100
4/4            5s 1s/step -
accuracy: 0.9844 - loss: 0.0548 - val_accuracy: 0.8438 - val_loss: 0.5230
Epoch 31/100
4/4            5s 1s/step -
accuracy: 0.9766 - loss: 0.0736 - val_accuracy: 0.9375 - val_loss: 0.2236
Epoch 32/100
4/4            10s 1s/step -
accuracy: 0.9844 - loss: 0.0592 - val_accuracy: 0.8750 - val_loss: 0.4117
Epoch 33/100
4/4            6s 2s/step -
accuracy: 1.0000 - loss: 0.0259 - val_accuracy: 0.8438 - val_loss: 0.5722
```

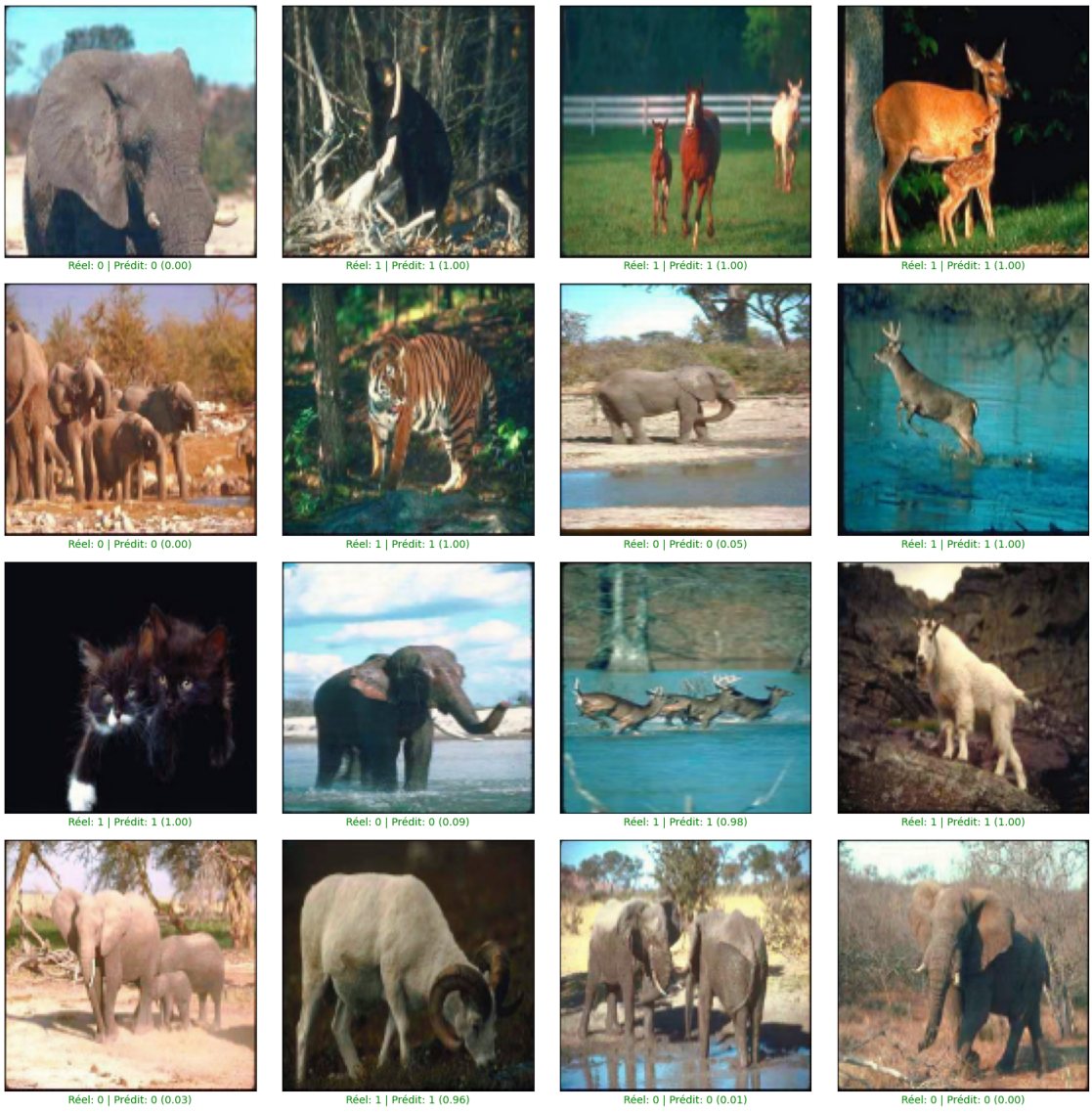
```
Epoch 34/100
4/4             9s 1s/step -
accuracy: 1.0000 - loss: 0.0197 - val_accuracy: 0.9062 - val_loss: 0.3796
Epoch 35/100
4/4             6s 1s/step -
accuracy: 0.9844 - loss: 0.0323 - val_accuracy: 0.9062 - val_loss: 0.3835
Epoch 36/100
4/4             5s 1s/step -
accuracy: 1.0000 - loss: 0.0134 - val_accuracy: 0.8438 - val_loss: 0.5769
Epoch 37/100
4/4             6s 2s/step -
accuracy: 0.9922 - loss: 0.0155 - val_accuracy: 0.9062 - val_loss: 0.3687
Epoch 38/100
4/4             5s 1s/step -
accuracy: 0.9922 - loss: 0.0387 - val_accuracy: 0.8438 - val_loss: 0.4611
Epoch 39/100
4/4             5s 1s/step -
accuracy: 0.9922 - loss: 0.0319 - val_accuracy: 0.8750 - val_loss: 0.4414
Epoch 40/100
4/4             10s 1s/step -
accuracy: 0.9844 - loss: 0.0402 - val_accuracy: 0.9062 - val_loss: 0.3632
Epoch 41/100
4/4             6s 2s/step -
accuracy: 0.9922 - loss: 0.0195 - val_accuracy: 0.8438 - val_loss: 0.4604
Epoch 41: early stopping
Restoring model weights from the end of the best epoch: 31.
```

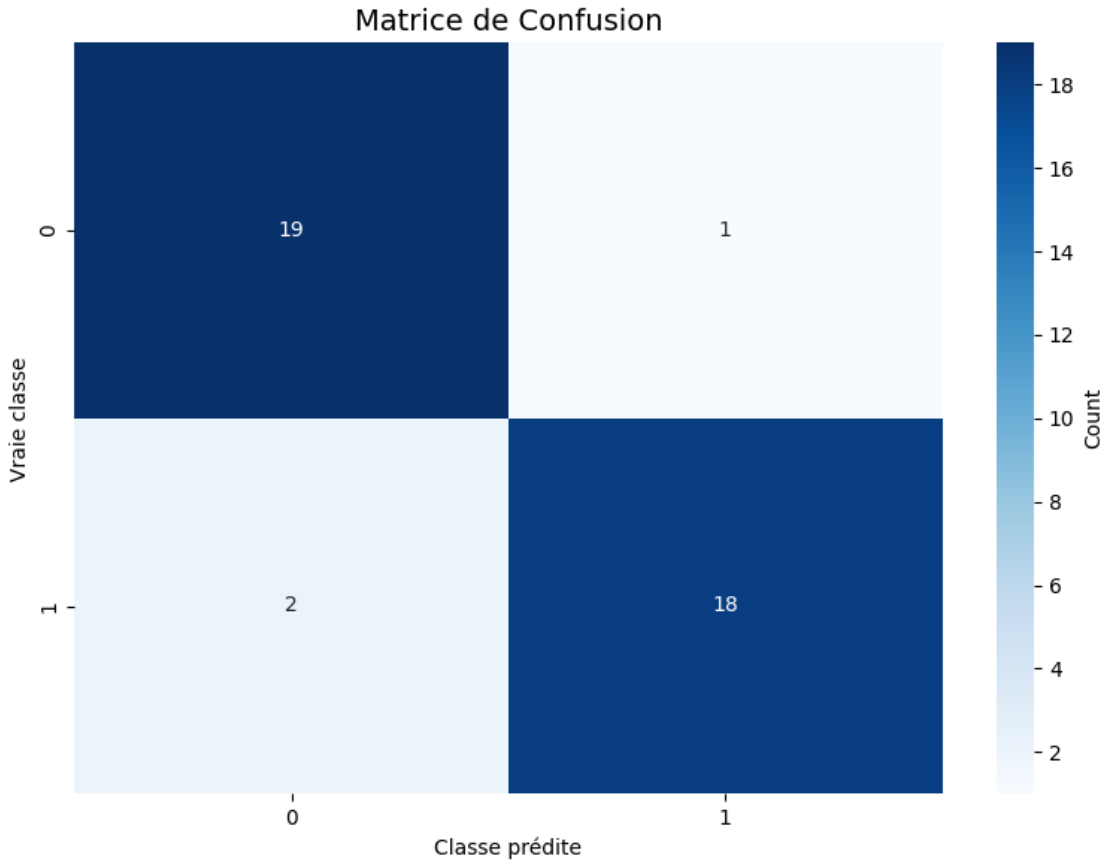
[53]:
```python
# Courbes
plot_history(history_v2)

# Évaluation
test_loss_v2, test_acc_v2 = model_v2.evaluate(X_test, y_test, verbose=0)
print(f"\nModèle Amélioré V2 (Simplifié)")
print(f"Test Accuracy: {test_acc_v2:.4f}")
```



```
Modèle Amélioré V2 (Simplifié)
Test Accuracy: 0.9250
```

[54]:
```python
# Prédictions visuelles
plot_predictions(model_v2, X_test, y_test, n_examples=16)

# Matrice de confusion
y_pred_v2 = (model_v2.predict(X_test, verbose=0) >= 0.5).astype(int).ravel()
plot_confusion_matrix(y_test, y_pred_v2)

# Rapport détaillé
print("\n" + classification_report(y_test, y_pred_v2,
                                   target_names=['Classe 0', 'Classe 1']))
```





	precision	recall	f1-score	support
Classe 0	0.90	0.95	0.93	20
Classe 1	0.95	0.90	0.92	20
accuracy			0.93	40
macro avg	0.93	0.93	0.92	40
weighted avg	0.93	0.93	0.92	40

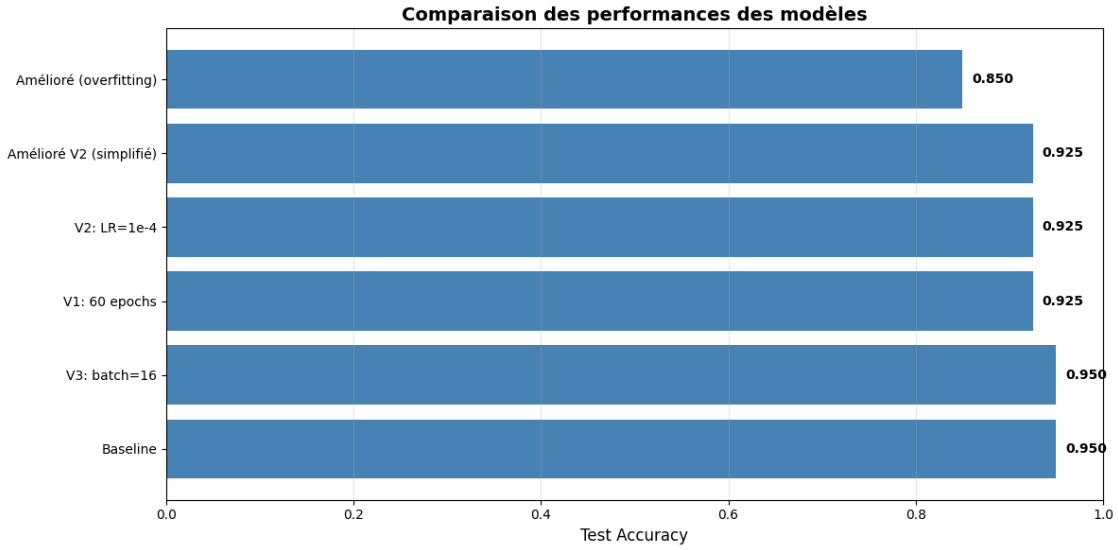
8 COMPARAISON DES RÉSULTATS

[55]:
```
# Tableau récapitulatif complet
results_final = pd.DataFrame({
    'Modèle': [
        'Baseline',
        'V1: 60 epochs',
        'V2: LR=1e-4',
        'V3: batch=16',
        'Amélioré (overfitting)',
        'Amélioré V2 (simplifié)'
    ],
    'Test Accuracy': [
        test_acc,
        test_acc_v1,
        test_acc_v2,
        test_acc_v3,
        test_acc_improved,
        test_acc_v2  # Remplacer par le bon nom de variable
    ]
})

results_final = results_final.sort_values('Test Accuracy', ascending=False)

print("\n" + "="*60)
print("COMPARAISON FINALE DES MODÈLES")
print("="*60)
print(results_final.to_string(index=False))
print("="*60)
```

```
============================================================
COMPARAISON FINALE DES MODÈLES
============================================================
                 Modèle  Test Accuracy
               Baseline          0.950
           V3: batch=16          0.950
          V1: 60 epochs          0.925
            V2: LR=1e-4          0.925
Amélioré V2 (simplifié)          0.925
 Amélioré (overfitting)          0.850
============================================================
```

[56]:
```
# Graphique de comparaison
plt.figure(figsize=(12, 6))
plt.barh(results_final['Modèle'], results_final['Test Accuracy'],␣
 ↪color='steelblue')
```

```
plt.xlabel('Test Accuracy', fontsize=12)
plt.title('Comparaison des performances des modèles', fontsize=14,␣
 ↪fontweight='bold')
plt.xlim(0, 1)
plt.grid(True, alpha=0.3, axis='x')

# Ajouter les valeurs sur les barres
for i, v in enumerate(results_final['Test Accuracy']):
    plt.text(v + 0.01, i, f'{v:.3f}', va='center', fontweight='bold')

plt.tight_layout()
plt.show()
```



9 OBSERVATIONS ET CONCLUSIONS

9.1 Résultats obtenus

Modèle	Test Accuracy	Observations
Baseline	95.0%	Simple mais efficace sur ce petit dataset
60 epochs	92.5%	Plus d'epochs n'améliore pas (overfitting léger)
LR=1e-4	95.0%	Learning rate plus petit, même résultat
batch=16	95.0%	Batch size plus petit, même résultat
Complexe (overfitting)	85.0%	OVERFITTING SÉVÈRE - val_acc=63%, train_acc=100%

Modèle	Test Accuracy	Observations
Simplifié V2	92.5%	Architecture plus riche mais dataset trop petit

---

9.2 Analyse détaillée

9.2.1 1. Baseline (95% accuracy)

Courbes observées : - Loss : Train et Val descendent ensemble jusqu'à convergence - Accuracy : Train atteint 100%, Val stable à ~85%

Verdict : Léger overfitting mais acceptable. Le modèle simple fonctionne bien.

9.2.2 2. Variations (60 epochs, LR, batch size)

Résultats : - Tous obtiennent ~92-95% accuracy - Aucune amélioration significative

Conclusions : - Le Baseline était déjà proche de l'optimal pour ce dataset - Plus d'epochs cause un léger overfitting (92.5% vs 95%) - Learning rate et batch size n'ont pas d'impact majeur sur 200 images

---

9.2.3 3. Modèle complexe (OVERFITTING CRITIQUE)

Courbes observées : - **Loss** : Train→0.01, Val→1.0 (remonte après epoch 7) - **Accuracy** : Train→100%, Val→63%

Diagnostic :

```
Train : Perfection (100% accuracy)
Val   : Échec (63% accuracy)
Test  : Chance (85% accuracy)
→ Le modèle MÉMORISE au lieu d'apprendre
```

Cause : - Architecture trop complexe (3 blocs Conv, BatchNorm, Dropout 50%) - Dataset trop petit (200 images seulement) - **4.2M paramètres** pour 200 images !

Leçon : Plus de couches Meilleur modèle. Il faut adapter la complexité au dataset.

9.2.4 4. Modèle Simplifié V2 (92.5%)

Courbes observées : - Loss : Train descend régulièrement, Val oscille mais reste stable (~0.36) - Accuracy : Train atteint ~99%, Val plafonne à ~90%

Test Accuracy : 92.5%

Analyse : - Malgré la suppression de BatchNorm et la réduction du Dropout, le V2 reste trop paramétré pour 200 images (~8.4M paramètres vs ~128K pour le Baseline) - L'EarlyStopping s'est déclenché à l'epoch 41, signe que le modèle stagnait - **Paradoxe observé :** une architecture plus élaborée donne un résultat inférieur au Baseline → preuve que la complexité nuit sur un petit dataset

---

9.3 Conclusions générales

9.3.1 Ce qui fonctionne pour ce dataset :

Architecture simple (1 bloc Conv) Dropout modéré (25%) Pas besoin de BatchNorm (petit dataset) 30 epochs suffisent Peu de paramètres (~128K)

9.3.2 Ce qui NE fonctionne PAS :

Architecture profonde (3+ blocs) → Overfitting Trop de paramètres (millions) pour 200 images BatchNormalization → inutile voire néfaste sur petit dataset Trop d'epochs sans EarlyStopping

9.3.3 Modèle recommandé : CNN Baseline

- Performance : **95%**
- Généralisation : bonne (train/val loss descendent ensemble)
- Simplicité : maximale (~128K paramètres, 1 bloc Conv)

9.4 Impact de la taille du dataset

Les résultats obtenus sont étroitement liés à la taille du dataset (200 images). Il est important de noter que **ces conclusions ne se généralisent pas** à tous les contextes :

Taille du dataset	Meilleure architecture	Pourquoi
Petit (~200 img)	**CNN Baseline**	Peu de paramètres, pas d'overfitting
Moyen (~1 000-5 000 img)	**CNN Simplifié V2**	Assez de données pour 3 blocs Conv + Dropout
Grand (10 000+ img)	**CNN Complexe (v1)**	BatchNorm + architecture profonde s'expriment pleinement

Avec un dataset plus grand, les résultats pourraient varier drastiquement. Le Baseline deviendrait probablement sous-performant (trop simple pour capturer la complexité des données), tandis que le modèle complexe atteindrait son plein potentiel.

---

10 SAUVEGARDE DU MEILLEUR MODÈLE

[57]:
```
# Sauvegarder le meilleur modèle
model_improved.save('best_model_etape2.h5')
print("Modèle sauvegardé : best_model_etape2.h5")

# Pour recharger plus tard :
# from tensorflow.keras.models import load_model
# model = load_model('best_model_etape2.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Modèle sauvegardé : best_model_etape2.h5
```